From Android Engineer to AI Software Engineer

Your practical transition roadmap

Python backend • RAG • agents • Android + AI

Designed for an experienced Android / Software Engineer currently on bench. Turn existing engineering experience into evidence that you can build and operate useful GenAI applications.

Duration	Working rhythm	Proof of progress
12 weeks, flexible start	14-16 hours/week, including career work and catch-up	3 connected projects, evaluation reports, deployed demo and interview stories

Name: _____ Start date: _____

Target roles: AI Software Engineer / GenAI Engineer / Backend Engineer, AI products

Prepared 28 October 2026. This is a learning and job-search plan, not a hiring guarantee. Start with your current skill level; extend a week when its acceptance criteria are not yet met.

Contents and version notes

Pages	Section
1-4	Start here, time budget and default stack
5	12-week overview
6-29	Daily plans: two pages per week
30-32	Three portfolio project briefs
33-37	Quality, interviews, positioning, applications, certification
38-40	Fillable mentor review sheets
41	Fillable daily log
42	Resources

Corrected edition

Changes: evaluation split created before tuning; daily tasks separated into checkboxes; two-page weekly layouts; explicit core and optional scope; time reserved for catch-up and career work; concrete starter choices; certification exercises; fillable review fields.

The contents use printed PDF page numbers. Daily checkboxes and tracking fields can be saved in a compatible PDF reader. Keep a personal copy; viewer support varies.

How to use this roadmap

Build on your existing strengths

Keep your Android identity: API integration, architecture, asynchronous work, testing, lifecycle management and product delivery transfer directly. Your new focus is Python services, model behavior, retrieval quality and reliable tool execution. You are changing specialization, not restarting your career.

One shared budget: 14-16 hours/week

Block	Action
10 min	Recall and choose today's outcome.
20 min	Read only the documentation needed for the task.
70 min	Implement one bounded slice.
20 min	Verify, commit and record evidence.

Days 1-5: 2 hours/day. Day 6: up to 2 hours for catch-up and shipping. Day 7: 30-minute review, then rest. Reserve another 1.5-3.5 hours for applications, interview practice and later certification. Total: 14-16 hours. These activities share one budget; they are not extra homework.

Minimum viable day: 45 minutes

Spend 5 minutes recalling, 30 minutes building one small change, and 10 minutes checking and logging it. If bench ends or client work resumes, use 8-10 hours/week and stretch each roadmap week across two calendar weeks.

Mentor check-in

Return with: Week/Day; time spent; completed checklist; commit/demo link; one test or metric; blocker and exact error; what you can explain without help. Ask for a review and the next task. A mentor can adjust the plan when you check in; this PDF does not schedule reminders.

Weekly rule

Day 6 ships evidence. Day 7 is a 30-minute review plus rest or a short catch-up. Complete the core acceptance criteria before optional features. Track achievements only when the evidence exists.

Default stack and scope boundaries

Area	Default path for this learning plan
Python/backend	One supported Python version; venv + pip, FastAPI, Pydantic, httpx and pytest. Pin working dependencies.
Persistence	PostgreSQL + pgvector; SQLAlchemy + Alembic for data access and migrations. Qdrant is optional later.
Models	Claude for generation and tools. Use a small sentence-transformers embedding model locally; record its exact ID and revision. OpenAI adapter is optional.
Authentication	For the private learning demo, follow FastAPI OAuth2/password + JWT tutorial with password hashing and two synthetic users. No public signup. Reassess auth before real users.
Agent/MCP	LangGraph for stateful workflows; official Python MCP SDK for two local read-only tools via stdio. Remote HTTP deployment is optional.
Deployment	Docker Compose locally. Default staging path: Render container service plus managed PostgreSQL, after checking current pricing and pgvector support. Local video demo is the no-hosting fallback.
Observability	Start with structured JSON logs and correlation IDs. Add OpenTelemetry traces if time remains; choose one viewer, not several.
Android	Kotlin + Compose + coroutines, calling the same FastAPI backend. Ship a focused client.

Core versus optional

Core: one provider, one corpus, vector retrieval, source-linked answers, owner isolation, measured evaluation, one bounded agent, local MCP tools and an Android client. Optional: provider comparison, hybrid retrieval, reranking, streaming polish, public hosting and advanced telemetry.

Day 6 adds no new topic. Finish the acceptance gate or use the reduced fallback stated for the week. If a core gate still fails, extend the week. Keep quality and permission checks; cut features first.

Your 12-week map

Week	Focus	Evidence / achievement
1	Python foundations	Python Environment Ready
2	FastAPI backend	AI-Ready Backend
3	LLM APIs and tool calling	LLM Integrated
4	RAG: ingestion and retrieval	Knowledge Indexed
5	RAG: answers and citations	Enterprise RAG v1
6	Evaluation and RAG hardening	RAG Quality Measured
7	LangGraph and agent workflows	Agent Workflow Built
8	Agent reliability and observability	Reliable Agent v1
9	MCP integrations	MCP Connected
10	Docker and deployment	AI Service Deployed
11	Android + AI and portfolio	Android + AI Shipped
12	Interviews and applications	Interview-Ready Portfolio

Scope discipline

Enterprise RAG Assistant is the main project. Agentic AI Platform extends it with a separate workflow module. Android + AI is a client of the same backend. Reuse authentication, document storage, deployment and evaluations to keep three portfolio stories achievable.

Core stack: Python + FastAPI + PostgreSQL/pgvector + one model provider. Add LangGraph when stateful workflows need it, and MCP after local tools are reliable. Qdrant is a valid alternative to pgvector; learn one deeply before comparing another.

Week 01 | Python foundations

Build a typed, testable Python client and a clean repository.

Day 1 / Setup

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Check a maintained Python version supported by your libraries, Git, editor and Docker.
- [ ] Create genai-engineer-journey, a virtual environment, README and .gitignore.
- [ ] Run a hello script.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Types and collections

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Practice functions, lists/dicts, type hints and exceptions.
- [ ] Transform a small JSON payload into typed records; explain Kotlin-to-Python differences.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Models and structure

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Use dataclasses and Pydantic validation.
- [ ] Split config, client and models into modules; reject malformed input with a useful error.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 01 | Python foundations

Build a typed, testable Python client and a clean repository.

Day 4 / HTTP and configuration

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Call a public JSON API with a timeout.
- [ ] Load configuration from environment variables; add .env.example with placeholders and keep secrets out of Git.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Async and tests

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Use an async HTTP client for two independent requests.
- [ ] Write pytest cases for success, timeout and invalid data using mocks.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Make a CLI that fetches and summarizes public data. Add setup/run instructions and a sample output; reproduce it in a fresh environment.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

A fresh checkout runs the CLI; at least 3 meaningful failure/success tests pass; you can explain await, validation and exception flow.

Fallback: Keep one public API and a CLI; omit extra libraries.

Achievement: Python Environment Ready. Award only after passing the core gate.

Week 02 | FastAPI backend

Build the backend foundation used by all three portfolio projects.

Day 1 / Endpoints

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Create /health and CRUD endpoints for documents metadata.
- [ ] Define request and response models; inspect generated OpenAPI.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Persistence

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Add PostgreSQL and migrations.
- [ ] Store document IDs and ownership; avoid placing database logic directly in routes.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Private demo authentication

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Follow the FastAPI JWT/password tutorial with password hashing and two seeded synthetic users.
- [ ] Protect one endpoint and reject expired or invalid tokens.
- [ ] Keep public signup out of scope.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 02 | FastAPI backend

Build the backend foundation used by all three portfolio projects.

Day 4 / Authorization

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Scope queries to the authenticated owner.
- [ ] Verify that user B cannot read or delete user A's document by changing an ID.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Essential error handling

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Bound request inputs and return consistent errors.
- [ ] Test invalid input, missing login and cross-user access.
- [ ] Optional: pagination after these tests pass.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Document local setup and example requests. Add a CI test job; demonstrate create, list and forbidden access.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

CRUD persists across restart; protected routes reject unauthenticated access; cross-user tests pass; CI can run without real model credentials.

Fallback: Keep two seeded users and ownership tests; postpone pagination and registration.

Achievement: AI-Ready Backend. Award only after passing the core gate.

Week 03 | LLM APIs and tool calling

Create a small model-backed service before introducing orchestration.

Day 1 / First integration

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Use Claude as the primary provider.
- [ ] Read the current official SDK quickstart, put credentials on the server and implement /chat with a timeout.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Structured responses

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Define a schema for a support summary.
- [ ] Validate returned data; distinguish invalid output, refusal and transient provider failures.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Streaming and budgets

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Stream text to a simple client.
- [ ] Handle cancellation, token limits and bounded conversation history; record usage and elapsed time.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 03 | LLM APIs and tool calling

Create a small model-backed service before introducing orchestration.

Day 4 / Tools

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Implement two safe tools: search local FAQ and calculate a total.
- [ ] Validate arguments and return structured tool errors; never eval model text.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Bounded tool loop

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Execute requested tools and return results to the model.
- [ ] Limit steps and retries; test unknown tools, invalid arguments and tool timeout.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Finish the Claude endpoint and bounded tool loop; document setup and one failure example. Optional: compare 10 fixed prompts with an OpenAI adapter after the core gate passes.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Schema outputs validate on your fixture set; tool execution is bounded; errors are visible; no keys appear in code or logs. OpenAI comparison may be mocked if unfunded.

Fallback: Keep one provider, validated output and two tools; postpone streaming and provider comparison.

Achievement: LLM Integrated. Award only after passing the core gate.

Week 04 | RAG: ingestion and retrieval

Start Enterprise RAG Assistant with traceable source documents.

Day 1 / Dataset and split

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Collect 20 public or synthetic policy documents.
- [ ] Label 40 questions with source IDs: 28 answerable, 8 unanswerable and 4 adversarial.
- [ ] Save 25 development cases (18/5/2) and 15 held-out cases (10/3/2) before any tuning; keep held-out questions out of experiments.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Parsing

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Extract text and preserve title, source, section/page and owner metadata.
- [ ] Log parsing failures; exclude unreadable or empty files.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Chunking

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Implement a simple section-aware chunker.
- [ ] Try two chunk sizes and overlaps; inspect 10 chunks manually for context loss.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 04 | RAG: ingestion and retrieval

Start Enterprise RAG Assistant with traceable source documents.

Day 4 / Embeddings and storage

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Choose one embedding model and PostgreSQL + pgvector.
- [ ] Store model/version metadata; batch embeddings and upsert stable chunk IDs.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Retrieval

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Build top-k search with authorization filters.
- [ ] Return source IDs and scores; inspect results for 10 labeled questions before generation.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Make ingestion repeatable and duplicate-safe. Add delete/reindex behavior and document the pipeline; keep evaluation fixtures versioned.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Every chunk resolves to a source; re-ingestion avoids duplicates; deleting a document removes retrievable chunks; retrieval cannot cross owner boundaries.

Fallback: Use text/Markdown documents first; postpone OCR and complex PDF parsing.

Achievement: Knowledge Indexed. Award only after passing the core gate.

Week 05 | RAG: answers and citations

Turn retrieval into a trustworthy assistant and publish a first demo.

Day 1 / Grounded answers

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Build /ask using retrieved context.
- [ ] Require citations by chunk ID and an explicit insufficient-evidence response.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Citation validation

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Resolve citations to original source locations.
- [ ] Reject unknown citation IDs; inspect whether each cited passage actually supports its claim.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Baseline comparison

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Measure vector retrieval on development questions only.
- [ ] Inspect its misses and record recall@5 with a source-ID matching rule.
- [ ] Optional: compare lexical or hybrid retrieval if the baseline is already working.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 05 | RAG: answers and citations

Turn retrieval into a trustworthy assistant and publish a first demo.

Day 4 / One improvement

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Fix the largest development-set retrieval failure.
- [ ] Save before/after results using the same configuration record.
- [ ] Optional: evaluate a reranker and its extra latency/cost.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Safety and UI

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Treat document text as untrusted data.
- [ ] Test injected instructions, unsupported questions and cross-user access; add a minimal chat UI with source previews.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Record a 2-3 minute demo: normal answer, source inspection, abstention and an attack attempt. Write the design decisions and current limitations.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Demo handles answerable and unanswerable questions; citations open the right source; evidence and attack cases are recorded. Call it a prototype until operations are hardened.

Fallback: Keep vector search and source previews; postpone hybrid search and reranking.

Achievement: Enterprise RAG v1. Award only after passing the core gate.

Week 06 | Evaluation and RAG hardening

Measure the assistant and turn findings into an engineering report.

Day 1 / Audit the evaluation contract

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Verify that the week-4 held-out questions have not been used for tuning.
- [ ] Freeze the selected configuration and labels.
- [ ] If test questions were exposed, create a fresh independently labeled holdout and document the change.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Retrieval metrics

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Calculate recall@5 against expected evidence chunks or source IDs.
- [ ] Define the matching unit and inspect every miss.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Answer metrics

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Score correctness, citation support and abstention with a written rubric.
- [ ] Use human review for the small set; audit any model-judge scores.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 06 | Evaluation and RAG hardening

Measure the assistant and turn findings into an engineering report.

Day 4 / Final held-out run

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Compare candidate settings on development cases only and select one.
- [ ] Run baseline and selected configuration on the untouched held-out set once.
- [ ] Report counts and examples; any later fixes need a new test set for a fresh generalization claim.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Latency and cost

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Record per-request input/output usage, retrieval time, model time and total time.
- [ ] Run at least 30 timed requests; label p95 estimates as a small sample.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Finish the evaluation report with versions, raw counts and three failure analyses. Add development regression cases for fixes. Preserve the original held-out result; do not quietly retune it.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Publish baseline versus selected configuration, honest held-out results and at least 3 failure analyses. Small benchmark results are evidence, not a universal accuracy claim.

Fallback: Keep human scoring on the small fixture set; postpone automated model judges.

Achievement: RAG Quality Measured. Award only after passing the core gate.

Week 07 | LangGraph and agent workflows

Start Agentic AI Platform as a bounded workflow, reusing RAG.

Day 1 / Choose the workflow

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Define a support triage flow: classify issue, retrieve policy, draft response, request approval, create a sandbox ticket.
- [ ] Compare it with a deterministic pipeline.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / State and nodes

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Implement typed state and LangGraph nodes.
- [ ] Track request ID, tool results and status; make routing conditions explicit.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Tool contracts

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Add retrieve_policy, lookup_ticket and create_sandbox_ticket.
- [ ] Validate schemas and permissions; use synthetic data and sandbox integrations.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 07 | LangGraph and agent workflows

Start Agentic AI Platform as a bounded workflow, reusing RAG.

Day 4 / Checkpointing

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Persist state and resume after interruption.
- [ ] Simulate a crash between steps; inspect what is re-executed.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Approval and idempotency

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Pause before ticket creation.
- [ ] Bind approval to the exact proposed action; add an idempotency key to prevent duplicate side effects.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Demonstrate successful routing, denial and resume. Add a trace showing each state transition; explain why a graph was justified.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

The workflow resumes from stored state; writes require explicit approval; replay does not create duplicate tickets; deterministic alternatives are documented.

Fallback: Use one deterministic route plus one model decision; avoid multiple agents.

Achievement: Agent Workflow Built. Award only after passing the core gate.

Week 08 | Agent reliability and observability

Make agent failures controlled, diagnosable and measurable.

Day 1 / Budgets

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Add maximum steps, wall-clock deadline, tool concurrency limit and per-run cost guard.
- [ ] Show a clear stop reason when a budget is hit.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Recovery

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Retry transient reads with backoff.
- [ ] Do not blindly retry writes; test partial failure, duplicate requests and provider outage.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Traces

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Capture graph node, tool name, latency, status and correlation ID.
- [ ] Redact sensitive inputs and store only necessary content.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 08 | Agent reliability and observability

Make agent failures controlled, diagnosable and measurable.

Day 4 / Scenario evaluation

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Create 20 fixed scenarios: normal paths, malicious tool text, unauthorized access, timeout, rejected approval and repeated requests.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Inspect and improve

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Compare scenario success and tool-call correctness before/after one change.
- [ ] Separate model mistakes from orchestration and permission failures.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Expose create-run, run-status and resume endpoints. Publish a demo and architecture diagram with failure handling and limitations.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

All critical authorization/approval cases pass; 20-scenario report includes failures; every run has a trace and final status; budgets reliably terminate loops.

Fallback: Keep JSON traces and 20 scenarios; postpone dashboards.

Achievement: Reliable Agent v1. Award only after passing the core gate.

Week 09 | MCP integrations

Connect the agent to external tools through a small, secure MCP server.

Day 1 / Protocol basics

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Learn host, client and server roles; distinguish tools, resources and prompts.
- [ ] Trace one request using a local MCP development client.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Server

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Build a local Python MCP server exposing search_policy and get_ticket with typed inputs and structured results.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Client integration

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Connect your existing agent to this server.
- [ ] Reuse tool validation and limits; handle missing server and tool errors gracefully.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 09 | MCP integrations

Connect the agent to external tools through a small, secure MCP server.

Day 4 / Trust boundaries

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Allowlist tools and executable paths, minimize scopes and validate outputs.
- [ ] Keep approvals for writes; distinguish local stdio from remote HTTP auth needs.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Security exercise

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Test malicious tool descriptions/results, unauthorized IDs and server disconnect.
- [ ] For remote HTTP, study official authorization guidance; do not pass unrelated tokens through.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Publish setup, transport, permissions and a short sequence diagram. Record a working agent-to-MCP demo; list remaining remote-production requirements.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Two MCP tools work end-to-end; denied data stays denied; failure handling is tested; README states transport and trust assumptions. Remote hosting is optional.

Fallback: Use local stdio with two read tools; postpone remote auth and hosting.

Achievement: MCP Connected. Award only after passing the core gate.

Week 10 | Docker and deployment

Operate the RAG and agent backend in a reproducible environment.

Day 1 / Containers

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Create a Docker image and Compose setup with PostgreSQL.
- [ ] Run as non-root, pin dependencies and keep secrets outside image layers.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / One staging deployment

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Prepare one Render container service and a managed PostgreSQL instance after checking cost and pgvector availability.
- [ ] Deploy with TLS and server-side secrets if funded.
- [ ] Otherwise record a reproducible local Docker demo and label hosting incomplete.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Security checks

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Set upload/type/size limits, rate limits, restricted CORS and safe outbound destinations.
- [ ] Test auth, document ownership and approval on the deployed service.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 10 | Docker and deployment

Operate the RAG and agent backend in a reproducible environment.

Day 4 / Minimum operations

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Add request IDs, structured error logs and health/readiness endpoints.
- [ ] Record request latency and model usage with sensitive content redacted.
- [ ] Optional: connect an OpenTelemetry viewer and alerts.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Recovery

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Run migrations; test backup/restore on disposable data and rollback to a previous image.
- [ ] Set spend alerts and a hard app-side request budget.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Finish the deployment runbook and smoke test. Record backup/restore and previous-image rollback evidence from local disposable infrastructure if staging time is limited. State exactly which environment was tested.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

A reviewer can run Docker locally; staging is reachable or demonstrated privately; restart preserves data; backup restore and rollback are exercised.

Fallback: A local reproducible release is acceptable evidence; mark public hosting unfinished.

Achievement: AI Service Deployed. Award only after passing the core gate.

Week 11 | Android + AI and portfolio

Use Android experience to make your profile distinctive.

Day 1 / Scope the mobile client

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Build a Kotlin/Compose client for your existing RAG API: login, question entry, answer and source list.
- [ ] Reuse the backend instead of creating a third AI service.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / Async UX

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Use coroutines and lifecycle-aware state.
- [ ] Support loading, cancellation, retry and clear network errors; handle streamed output if time allows.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Mobile security

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Keep model API keys on the backend.
- [ ] Store session credentials appropriately; clear user state at logout and avoid logging sensitive answers.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 11 | Android + AI and portfolio

Use Android experience to make your profile distinctive.

Day 4 / Device validation

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Test slow network, rotation, expired login and long answers.
- [ ] Check accessibility and source navigation; record emulator/device results.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Portfolio essentials

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Update existing README files with verified setup and current limitations.
- [ ] Add one architecture diagram and one evaluation table per backend project.
- [ ] Optional: create a polished portfolio website later.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Finish the Android build and record a 90-second demo. Verify all three project links and capture one mobile error case. Use the career block for CV updates instead of adding a separate full workday.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

Android APK or reproducible build, a short video and a mobile failure-case checklist exist; all three projects have readable documentation and real evidence.

Fallback: Reuse the existing backend; postpone voice, offline AI and Play Store release.

Achievement: Android + AI Shipped. Award only after passing the core gate.

Week 12 | Interviews and applications

Convert the portfolio into interviews while closing skill gaps.

Day 1 / Coding mock

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Practice Python collections, async pitfalls, HTTP and SQL.
- [ ] Complete one timed coding exercise and explain complexity and edge cases.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 2 / RAG system design

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Design a multi-tenant knowledge assistant on paper.
- [ ] Cover ingestion, access controls, retrieval, evaluation, latency, cost, deletion and failures.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 3 / Agent mock

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Explain state, checkpointing, approvals, idempotency and MCP boundaries.
- [ ] Debug one failing scenario from your own traces.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Career block / inside the weekly time budget

Weeks 1-4: 90 minutes for role research, CV basics and short drills. Weeks 5-9: 120 minutes for applications and one mock. Weeks 10-12: up to 210 minutes shared between applications, mocks and optional certification. Prioritize active interviews over certification.

Week 12 | Interviews and applications

Convert the portfolio into interviews while closing skill gaps.

Day 4 / Behavioral stories

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Prepare 5 STAR stories from real Android/software experience: ownership, incident, tradeoff, ambiguity and collaboration.
- [ ] Add one AI project learning story.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 5 / Applications

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Submit 5 tailored applications and seek 2 relevant referrals or conversations.
- [ ] Tailor the top CV bullets and include the most relevant project demo.
- [ ] Save a reproducible example or test result and note one decision in the daily log.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 6 / Catch-up and ship

Budget: 120 min = 10 recall + 20 learn + 70 build + 20 verify/log. Stop at the time limit.

- [ ] Finish the highest-priority unfinished core task; add no new topic.
- [ ] Review mock feedback, check the certification readiness gate and plan the next 4 weeks from feedback and application funnel data.
- [ ] Commit evidence and check the weekly acceptance gate.

Done: the checked outputs can be demonstrated from the saved commit. If blocked, record the exact failing step.

Day 7 / 30-minute mentor review, then rest

- [ ] Record hours, evidence, one lesson and the next action on the weekly review sheet.

Weekly deliverable / acceptance gate

You can demo without a script, explain your metrics and failures, complete two mocks and maintain a real application tracker. Continue applying beyond week 12.

Fallback: Choose two strong mocks and tailored applications; postpone optional certification.

Achievement: Interview-Ready Portfolio. Award only after passing the core gate.

Portfolio brief 1 | Enterprise RAG Assistant

A private company knowledge assistant that answers from approved documents and shows supporting sources. Build with public or synthetic material; never publish employer documents.

Minimum release

- [ ] Authenticated document ingestion, search and question-answer endpoints; source metadata survives parsing and chunking.
- [ ] PostgreSQL + pgvector, explicit ownership filters, repeatable indexing and deletion. Use a separate embedding provider/model if needed; budget embeddings independently from generation.
- [ ] Answers with validated source IDs, readable source previews and abstention when evidence is insufficient.
- [ ] A reproducible 40-question benchmark split before tuning, baseline comparison, adversarial cases and per-request usage/latency.

Suggested structure

Client -> authenticated FastAPI -> authorized retrieval -> context builder -> Claude -> validated answer + citations. Ingestion: upload -> parser -> chunks -> embeddings -> vector index. Enforce access controls in retrieval and source endpoints, not only in prompts.

Suggested project targets, not promised results

Aim for recall@5 >= 0.80 on answerable held-out questions, >= 90% supported citations in a manual audit and correct abstention on all 3 held-out unanswerable cases as a small-sample learning target. Report raw counts and denominators; small test sets have large uncertainty. If you miss a target, publish the result and diagnosis.

Reviewer evidence

README; architecture diagram; dataset provenance; labeled fixtures; evaluation command and report; demo video; failure analysis; threat model. Show one unsupported question and one attempted prompt injection during the demo.

Stretch only after core completion

Hybrid retrieval, reranking, incremental indexing or multilingual questions. Fine-tuning and Kubernetes are outside the 12-week core scope.

Portfolio brief 2 | Agentic AI Platform

A support workflow that reads a request, checks policy, proposes a response and creates a sandbox ticket only after approval. A small dependable platform is enough.

Minimum release

- [ ] LangGraph workflow with typed state, routing and durable checkpoints; endpoints to start, inspect and resume runs.
- [ ] Three local tools with explicit input/output contracts; two exposed via MCP in week 9.
- [ ] Approval tied to the exact write arguments; owner-scoped permissions; idempotency keys and a visible audit trail.
- [ ] Step/time/cost budgets, tool timeouts, safe retry policy and recoverable failure states.
- [ ] 20 scenario fixtures covering success, denial, outage, malicious tool output, resume and repeated requests.

Evaluation contract

Report completed-correct / total scenarios, tool selection/argument correctness, duplicate side effects and approval bypass attempts. Require zero unauthorized writes and zero duplicate tickets in the defined safety fixtures before demoing write actions. These checks do not prove general security.

Architecture decisions to defend

Why a graph instead of a simple function chain? What state is persisted? Which steps can replay? What does approval authorize? What happens if a tool succeeds but its response is lost? Why does an MCP connection not grant blanket permission?

Reviewer evidence

State diagram, sanitized trace, scenario report, checkpoint/resume demo, tool contracts, approval audit example and a known-limitations section. Use sandbox writes; add real integrations only after explicit authorization and appropriate access controls.

Portfolio brief 3 | Android + AI

An Android knowledge companion using your existing backend. This is the bridge between your proven mobile experience and your new AI engineering skills.

Minimum release

- [ ] Kotlin/Compose screen for question entry, answer display, source references and session state.
- [ ] Lifecycle-aware coroutines, cancellation, loading/error states and readable long answers.
- [ ] Backend-issued authentication; no provider credentials in the APK; clear logout and session-expiry behavior.
- [ ] Network and UI checks covering rotation, slow response, offline state, repeated tap and unauthorized source access.
- [ ] APK or build instructions, screenshots and a 90-second demo that shows both value and recovery from an error.

Keep the backend as the authority

The mobile app sends the user request and session token. The backend decides document access, model usage, retrieval and rate limits. Hide privileged operations from the mobile client and validate every request server-side.

What makes this credible

Explain streaming versus full-response tradeoffs, mobile cancellation versus server job lifetime, session expiry and source display. Show how your Android architecture and testing skills improved the AI product experience.

Avoid expanding the scope

Voice, on-device inference, offline retrieval and a full Play Store release are optional future work. A focused, reliable client demonstrates more than several unfinished AI features.

Quality, security and operations scorecard

Area	Release evidence
Evaluation	Version prompts/model/config; keep held-out fixtures separate; compare against a simple baseline; inspect failures manually.
Authorization	Two-user access tests for documents, retrieval, source links, agent runs and resume endpoints.
Prompt injection	Treat retrieved text and tool results as data; test hostile instructions; enforce tool permissions in code.
Secrets and privacy	No keys in repo, APK, image or logs; use synthetic data; redact traces; document deletion and retention.
Tool safety	Validate arguments and destinations; allowlist tools; approve writes; use idempotency and bounded execution.
API resilience	Timeouts, bounded retries, request limits, cancellation and clear provider error behavior.
Observability	Request/run ID, node/tool name, status, latency, usage and estimated cost; avoid storing full sensitive prompts by default.
Deployment	TLS, managed secrets, pinned dependencies, health checks, database migrations and tested backup/rollback.

Budget before experiments

Set a personal weekly spending cap before real calls. Estimate: model input tokens x current input rate + output tokens x output rate + embeddings + reranking + hosting/storage. Rates differ by provider and model; check live pricing. Start with a tiny dataset, mock calls in CI and stop app-side requests when the cap is reached.

A useful experiment log

Date | commit | dataset/version | prompt/model/version | change | quality counts | latency sample size | estimated cost | failure examples | keep/revert decision.

Interview preparation | weekly practice

When	Practice and evidence
Weeks 1-2	Two 20-minute Python/SQL drills per week. Explain typing, mutable defaults, async vs threads, validation, dependency injection and joins.
Weeks 3-4	Explain the model request lifecycle, structured output, tool calling, context budgets, embeddings and retrieval. Draw the RAG pipeline from memory.
Weeks 5-6	One 30-minute RAG design mock each week. Defend chunking, recall@k, groundedness, abstention and leakage-free evaluation.
Weeks 7-9	One agent debugging/design mock each week. Cover state, retries, idempotency, approvals, tool trust and MCP transport/auth boundaries.
Weeks 10-12	One coding mock + one system design/behavioral mock per week. Record answers and act on the weakest area.

Questions you should be able to answer

When would you avoid an agent? How do you know a RAG change improved quality? Can a citation be valid but misleading? What happens when the model or vector DB is unavailable? How do you enforce tenant isolation? How do you prevent duplicate tool side effects? How would you reduce cost without losing acceptable quality?

Mock interview rubric: 0-2 per dimension

Score problem clarification, technical correctness, tradeoffs, failure/security handling and clarity. 0 = missing, 1 = partial, 2 = convincing with evidence. Target >= 8/10 across two mocks, with no zero in correctness or failure handling. This is a practice gate, not a prediction of hiring outcomes.

Behavioral preparation

Use real examples from Android/software delivery. For each STAR story, capture the constraints, your own contribution, the decision, the outcome and what you would change. Do not invent production AI experience from portfolio prototypes.

Positioning | CV and LinkedIn

Suggested headline

Software Engineer | Android + Python/FastAPI | Building RAG and agentic AI applications

Positioning summary to adapt

Software Engineer with Android product delivery experience, expanding into AI backend engineering. Building evaluated RAG services and controlled agent workflows with Python, FastAPI and Claude, with a focus on API reliability, access controls and mobile integration.

CV structure

Keep your real job titles and dates. Lead with relevant engineering achievements; add a clearly labeled AI portfolio/projects section. Put Python, FastAPI, RAG and evaluation above tools you have only sampled. Add demo and GitHub links near each project.

Evidence-backed bullet patterns

Built an authenticated RAG assistant over [N] public/synthetic documents using FastAPI and pgvector; achieved [measured recall@5] on [N] held-out answerable queries, with source-linked responses.

Implemented a checkpointed support agent with approval-gated writes and idempotency; passed [X/Y] defined scenarios, including replay, rejection and provider timeout.

Created an Android client for the AI backend with lifecycle-aware cancellation and source navigation; verified [specific device/network cases].

Replace every bracket with verified evidence or omit that claim. State sample size and test conditions; never imply prototype metrics came from customers.

How to discuss bench time

Describe it neutrally as a period between assignments used for focused upskilling and portfolio delivery. Keep employer status accurate. If permitted, seek an internal AI proof of concept or backend assignment alongside external applications.

LinkedIn content cadence

Post one useful artifact every 1-2 weeks: a demo, measured retrieval experiment, failure analysis or Android integration lesson. Link the evidence and explain one tradeoff. List certification only once earned; training completion is not certification.

Application timeline and tracking

When	Action
Week 1	Collect 10-15 relevant job descriptions. Extract repeated skills, seniority, language/location needs and evidence expected. Pick two target role families.
Weeks 2-3	Refresh CV basics and LinkedIn. Have 2 exploratory conversations/week with colleagues or recruiters; ask about actual project needs.
Weeks 4-5	Share the first RAG demo and explore internal opportunities. Start 2-3 targeted applications/week where your existing software background fits.
Weeks 6-8	Aim for 2-4 tailored applications/week and one short interview mock; include a relevant conversation when time permits. Lead with the RAG report; update links as the agent matures.
Weeks 9-12	Aim for 3-5 tailored applications/week when suitable roles exist; increase only if the time budget allows. Match each application to the strongest project; follow up after 7-10 business days when appropriate.
After week 12	Continue the pipeline. Every 2 weeks inspect response rates and feedback; improve the weakest evidence rather than adding more frameworks.

Application tracker columns

Company | role/link | fit notes | contact/referral | application date | CV version | project used | stage | follow-up date | feedback | next action.

Diagnose the funnel

Few replies: check role fit, CV clarity and demo accessibility. Recruiter screens but no technical progress: improve your transition story and fundamentals. Technical rejections: record the exact gap and practice it. Avoid drawing strong conclusions from a handful of applications.

Choose roles that value your bridge

Prioritize AI application/backend roles, product teams adding AI features and Android roles with AI integrations. Research-heavy ML roles may require deeper statistics, training and research experience beyond this roadmap. Preserve your existing software seniority while being candid about new AI skills.

Claude Architect certification | when and why

Recommended timing: assess readiness in week 10; consider sitting Claude Certified Architect - Foundations in weeks 12-14, after the core portfolio is usable. This timing is mentor guidance, not an official prerequisite.

Program information / reconfirm before booking

Earlier official-page results listed the Foundations Architect exam as 60 questions, 120 minutes and USD 125, with 12-month validity in the previously retrieved listing. Pearson VUE previously described access through organizations in the Claude Partner Network. A fresh page fetch was unavailable during this revision, so treat those details as provisional. Confirm eligibility, current exam guide, fee and delivery through the official links before paying. [1, 2]

Readiness gate

- [ ] You can build and explain a Claude API service, structured responses and a bounded tool loop.
- [ ] Your RAG evaluation report and agent approval/resume demo are complete.
- [ ] You can defend context management, reliability, MCP trust boundaries, cost and deployment choices.
- [ ] You have studied the current official blueprint, including Claude Code configuration/workflows, which the project plan alone does not fully cover.
- [ ] You can explain missed practice questions and complete two timed practice sessions without relying on memorized answers.
- [ ] Your eligibility, budget and booking route are confirmed through Anthropic Academy / Pearson VUE.

Concrete certification practice / optional career block

Week 10: 45 minutes to map the current blueprint to your evidence; 45 minutes to configure project instructions and scoped permissions in a disposable Claude Code repo. Week 11: 45 minutes for context management and a harmless hook/workflow exercise; 45 minutes to test scope boundaries and review changes. Week 12: two 45-minute timed scenario sessions plus 30 minutes reviewing mistakes. Use official available material; do not assume practice scores predict the exam.

Preparation allocation

Weeks 3-9: keep notes mapped to API, prompts, context, tools and agents. Weeks 10-12: reserve at most 90-120 minutes/week inside the shared career budget for blueprint gaps and official training; reduce optional project work. If more time is needed, add 1-2 weeks after the roadmap.

If access is unavailable or readiness is weak

Continue the portfolio and applications. Complete available official learning modules and accurately list them as training. Delay the exam instead of delaying your job search. Consider Developer Foundations only after comparing its current blueprint with target jobs; do not collect multiple credentials by default.

Official sources: [1] Anthropic Academy exam page; [2] Pearson VUE Claude program. Links on the resources page.

Mentor tracker | weeks 1-4

Fill in these fields in a compatible PDF reader and save your copy. A failed gate means adjust scope or extend the week, not skip the evidence.

Week 01 / Python Environment Ready

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 02 / AI-Ready Backend

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 03 / LLM Integrated

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 04 / Knowledge Indexed

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Mentor tracker | weeks 5-8

Fill in these fields in a compatible PDF reader and save your copy. A failed gate means adjust scope or extend the week, not skip the evidence.

Week 05 / Enterprise RAG v1

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 06 / RAG Quality Measured

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 07 / Agent Workflow Built

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 08 / Reliable Agent v1

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Mentor tracker | weeks 9-12

Fill in these fields in a compatible PDF reader and save your copy. A failed gate means adjust scope or extend the week, not skip the evidence.

Week 09 / MCP Connected

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 10 / AI Service Deployed

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 11 / Android + AI Shipped

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Week 12 / Interview-Ready Portfolio

☐ Core acceptance gate passed; evidence linked below.

Hours / evidence link / measurable result

Blocker / mentor feedback / next action

Daily log | reusable fillable sheet

Save a copy of this PDF for each review period, or duplicate this page in your PDF editor. Keep the detailed engineering log in your repository.

Week / day / date / planned minutes

One concrete outcome

- [ ] I built and can demonstrate the change.
- [ ] I checked one meaningful failure case.
- [ ] I saved evidence and chose the next task.

Commit or demo link / test result / sample size

What I learned / decision and reason

Blocker / what I tried / specific mentor question

Actual minutes / next smallest action

Resources and next steps

Use official documentation to answer the current implementation question, then return to the project. Avoid completing entire courses when one focused chapter is enough. Reference links collected 28 October 2026; availability may vary. Certification links could not be re-fetched for this revision.

Python tutorial - Weeks 1-2: syntax, modules, exceptions and classes.

FastAPI tutorial - Weeks 2 and 10: validation, dependencies, security and testing.

Claude tool use - Week 3: tool contracts and execution flow.

pgvector repository - Weeks 4-6: vector storage, indexing and query behavior.

LangGraph / LangChain OSS overview - Weeks 7-8: stateful orchestration and framework choice.

MCP documentation and security - Week 9: transport, authorization and tool trust boundaries.

Docker getting started - Week 10: images, containers and reproducible services.

Android architecture guidance - Week 11: lifecycle-aware mobile architecture.

[1] Anthropic Academy: Architect Foundations - Certification blueprint and preparation; verify current details.

[2] Pearson VUE: Claude certification program - Partner-network eligibility and exam scheduling information.

Your next action: Day 1

Create the repository, run a small Python script and write the first daily log. Bring the result to your mentor check-in. Your goal is a repeatable learning-and-shipping rhythm with visible evidence every week.